window, and go to http://localhost:9000.

Since Order Service doesn't have particular requirements depending on the user's role, you can log in with either Isabelle (`isabelle`/`password`) or Bjorn (`bjorn`/`password`). Then pick a book from the catalog, and submit an order for it. Since you're authenticated, you're allowed to create an order. When you're done, you can visit the Orders page to check all the submitted orders.

"Wait a second! What do you mean, all the submitted orders?" I'm glad you asked. At the moment, each person can see the orders submitted by all users. Don't worry! Later in the chapter, we'll fix that.

Before we do, though, we need to discuss how to test the new Order Service security policies. Stop the running applications (Ctrl-C) and containers (`docker-compose down`). The next section will show you how to test security in a reactive application.

12.3.2 Testing OAuth2 with Spring Security and Testcontainers

Testing secured reactive Spring Boot applications is similar to testing imperative applications. Before starting, we need to add new dependencies on Spring Security Test and Testcontainers Keycloak. The dependency adding JUnit5 support for Testcontainers is already there. Open the build.gradle file and update it as follows. Remember to refresh and reimport the Gradle dependencies after the new addition.

Listing 12.19 Adding dependencies to test Spring Security and Keycloak

```
ext {
  ...
```